



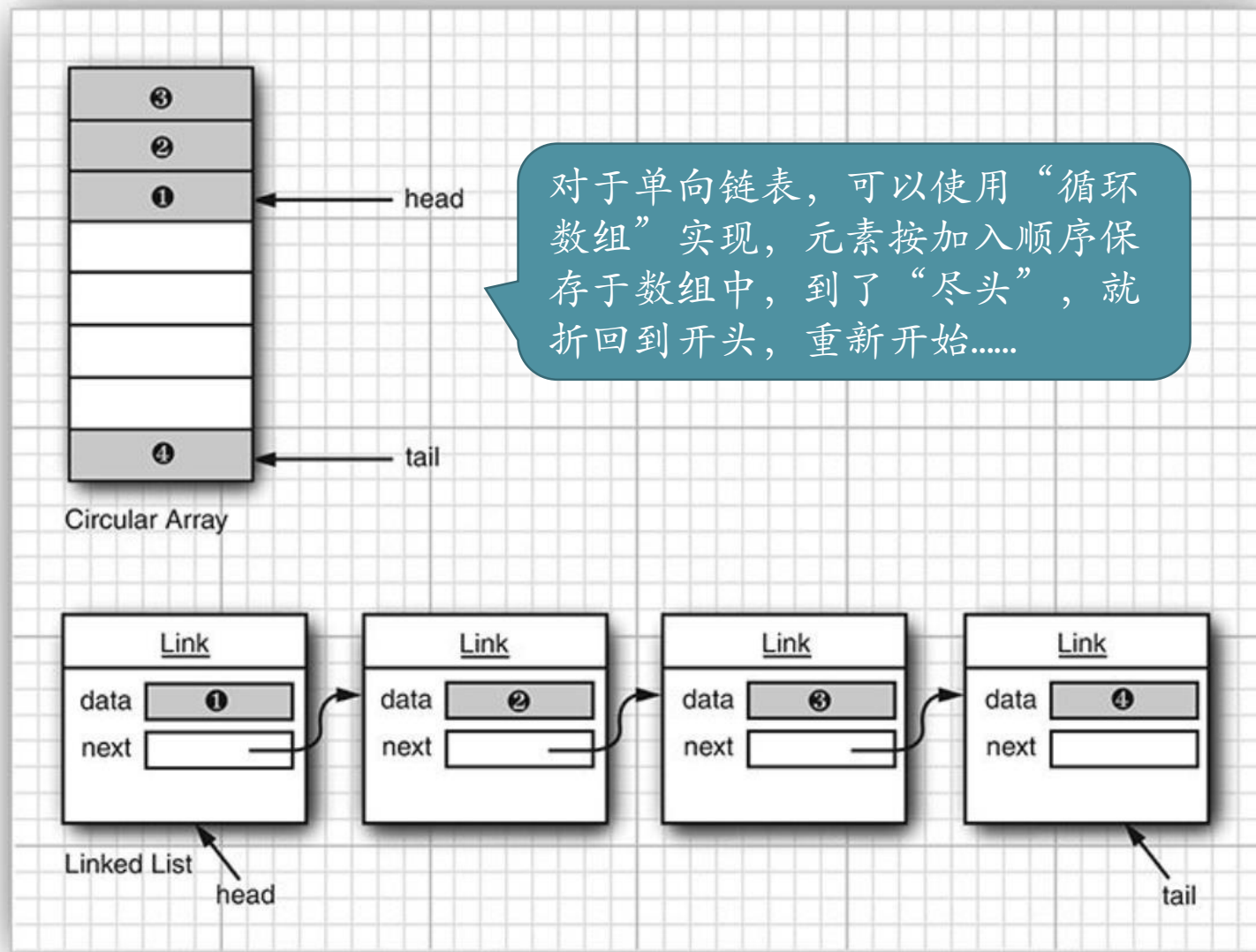
LinkedList

北京理工大学计算机学院
金旭亮

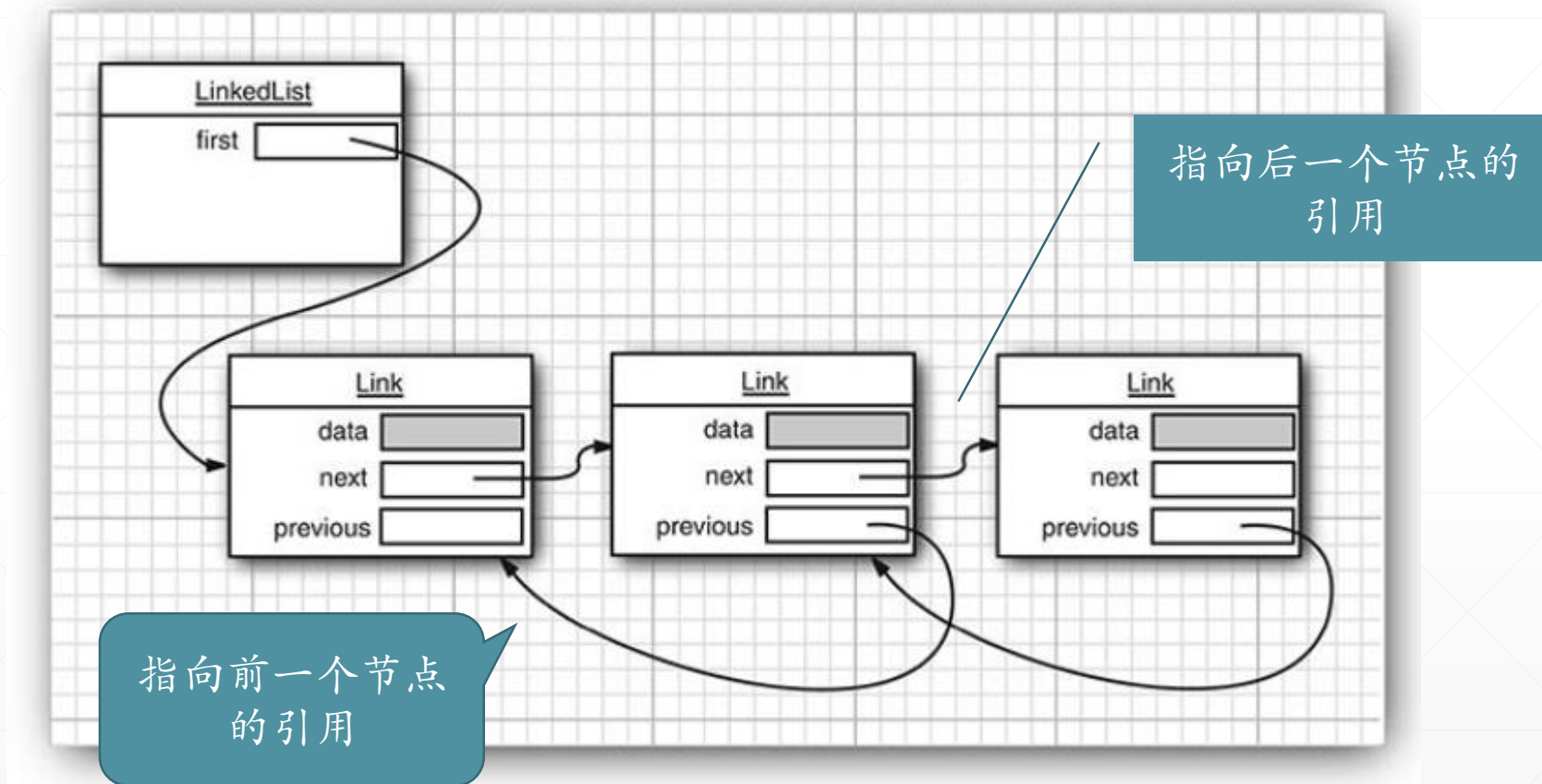
链表概述

链表是一种重要的数据结构，表现为一组先后连接的有顺序的数据块。

链表分为单向链表与双向链表两大类，对于单向链表来说，常用的实现方式有两种（如右图所示），双向链表通常使用第二种实现方式，给每个节点添加一个新的字段，引用前一个节点。

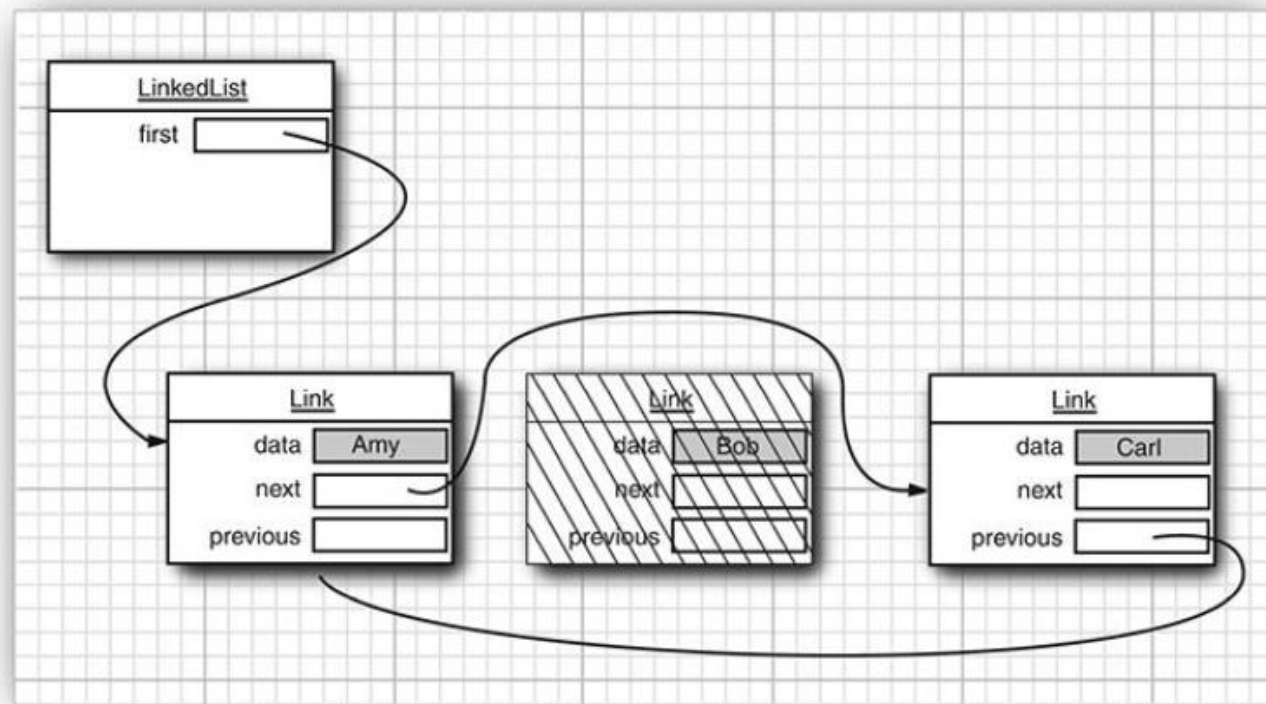


双向链表的典型实现方案



在JDK中，所有链表实际上都是双向链接的（doubly linked）——即每个节点存放着指向前驱节点的引用。

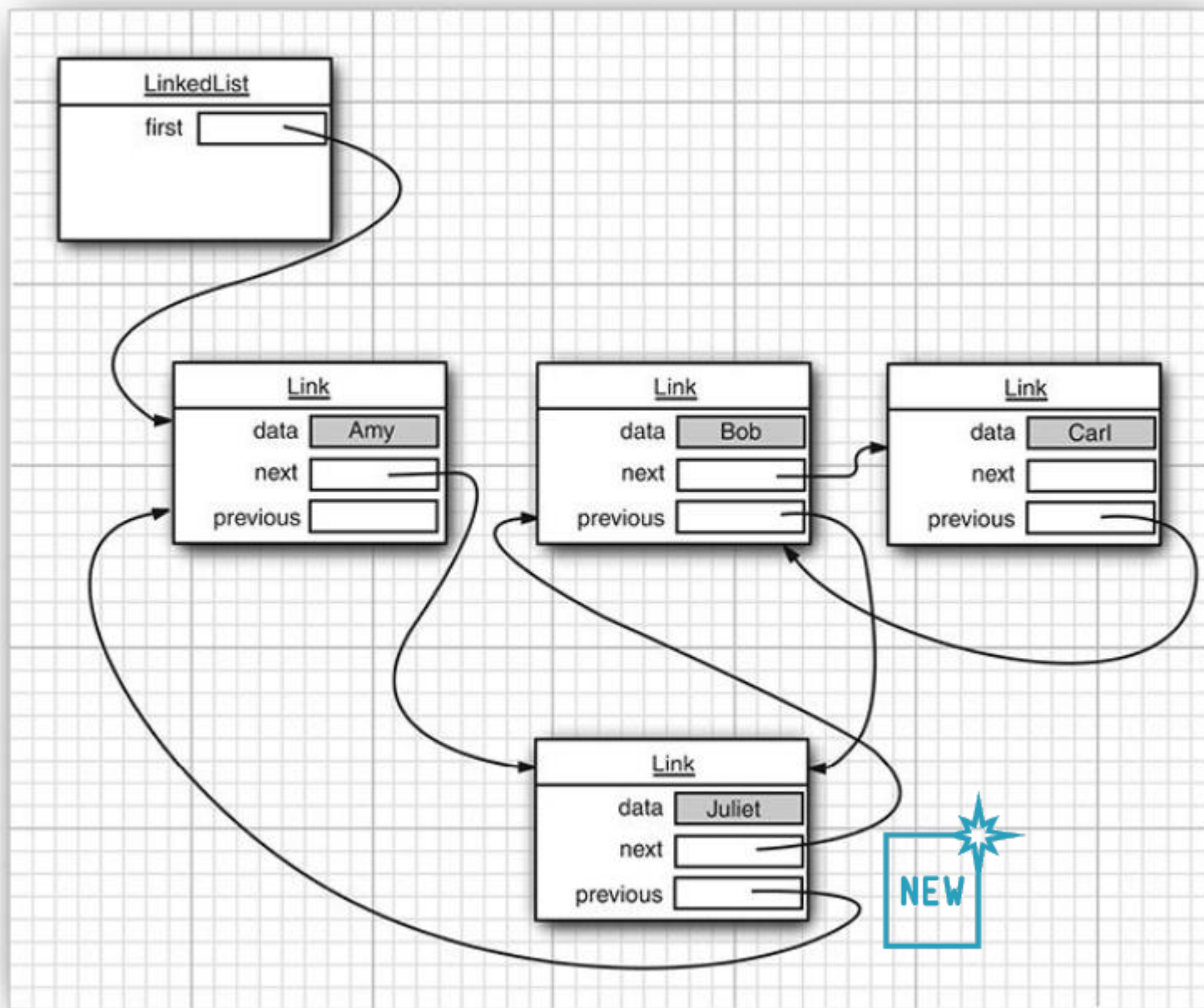
链表的基本操作—删除节点



从链表中移除一个元素，仅仅只需要修改它前后数据节点对象的引用就好，速度很快。

插入节点

向链表中添加一个节点，也只需要修改插入点前后两个对象的相应引用就好了，不会引发大规模的“数据搬家挪位置”的情况，速度也很快。

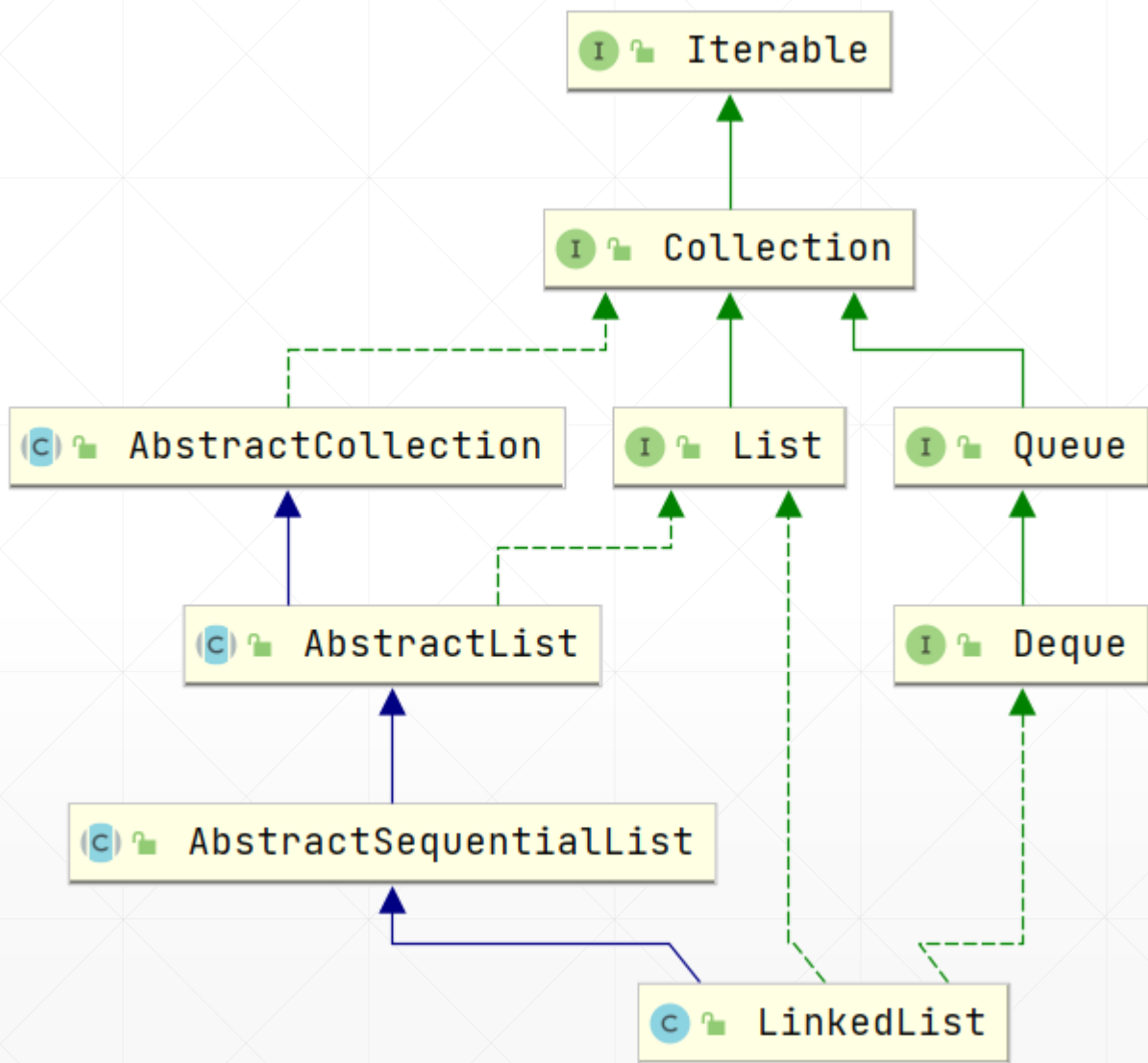


JDK中的LinkedList

LinkedList是基于链表实现的List类，对于顺序访问List中的元素进行了优化，插入/删除元素快，但随机访问（即指定要访问第XX个元素）性能不如ArrayList。

LinkedList同时实现了List和Deque接口，所以它既是一个List（有序集合），又是一个Deque（双端队列），还可以当作Stack（栈）来使用。

LinkedList: “我是一把瑞士军刀！”



LinkedList类的可用方法

LinkedList		
m	LinkedList()	
m	LinkedList(Collection<? extends E>)	
m	linkFirst(E)	void
m	linkLast(E)	void
m	linkBefore(E, Node<E>)	void
m	unlinkFirst(Node<E>)	E
m	unlinkLast(Node<E>)	E
m	unlink(Node<E>)	E
m	removeFirst()	E
m	removeLast()	E
m	addFirst(E)	void
m	addLast(E)	void
m	contains(Object)	boolean
m	size()	int
m	add(E)	boolean
m	remove(Object)	boolean

m	addAll(Collection<? extends E>)	boolean
m	addAll(int, Collection<? extends E>)	boolean
m	clear()	void
m	get(int)	E
m	set(int, E)	E
m	add(int, E)	void
m	remove(int)	E
m	isElementIndex(int)	boolean
m	isPositionIndex(int)	boolean
m	outOfBoundsMsg(int)	String
m	checkElementIndex(int)	void
m	checkPositionIndex(int)	void
m	node(int)	Node<E>
m	indexOf(Object)	int
m	lastIndexOf(Object)	int
m	peek()	E
m	element()	E
m	poll()	E
m	remove()	E
m	offer(E)	boolean
m	offerFirst(E)	boolean
m	offerLast(E)	boolean

m	peekFirst()	E
m	peekLast()	E
m	pollFirst()	E
m	pollLast()	E
m	push(E)	void
m	pop()	E
m	removeFirstOccurrence(Object)	boolean
m	removeLastOccurrence(Object)	boolean
m	listIterator(int)	ListIterator<E>
m	descendingIterator()	Iterator<E>
m	superClone()	LinkedList<E>
m	clone()	Object
m	toArray()	Object[]
m	toArray(T[])	T[]
m	writeObject(ObjectOutputStream)	void
m	readObject(ObjectInputStream)	void
m	splitIterator()	SplitIterator<E>
p	last	E
p	first	E

LinkedList示例

```
LinkedList<String> books = new LinkedList<>();
//将字符串元素加入队列的尾部
books.offer( e: "One book");
//将一个字符串元素入栈
books.push( e: "Two book");
//将字符串元素添加到队列的头部
books.offerFirst( e: "Three book");
System.out.println("LinkedList中的元素: ");

//LinkedList同时支持foreach迭代和传统的for循环遍历
//输出: Three book,Two book,One book,
for (String s : books) {
    System.out.print(s + ",");
}
System.out.println();
//输出: Three book|Two book|One book|
for (int i = 0; i < books.size(); i++) {
    System.out.print(books.get(i) + "|");
}
System.out.println();
//输出: [Three book, Two book, One book]
System.out.println(books);
```

TestLinkedList.java

```
//访问、并不删除队列的第一个元素
System.out.println("访问、并不删除队列的第一个元素:"
    + books.peekFirst());
//访问、并不删除队列的最后一个元素
System.out.println("访问、并不删除队列的最后一个元素:"
    + books.peekLast());
//采用出栈的方式将第一个元素pop出队列
System.out.println("采用出栈的方式将第一个元素pop出队列:"
    + books.pop());
//下面输出将看到队列中第一个元素被删除
System.out.println("弹出了一个元素后的LinkedList:" + books);
//访问、并删除队列的最后一个元素
System.out.println("访问、并删除队列的最后一个元素:"
    + books.pollLast());
//在队头与队尾添加元素
books.addLast( e: "Book: addLast");
books.addFirst( e: "Book: addFirst");
System.out.println("添加了新元素后的LinkedList: " + books + "\n");
//不断地从队尾移除
if (!books.isEmpty()) {
    books.removeLast();
}
System.out.println("LinkedList中的元素个数: " + books.size());
System.out.println("LinkedList中的元素: " + books);
```


试一试

请定义一个函数，它能交换LinkedList中任意两个节点的位置。

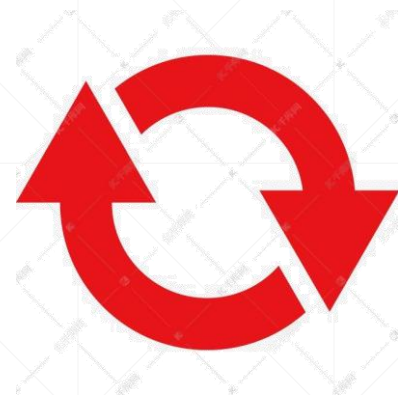
示例：

交换前：

A->B->**C**->D->**E**->F

交换后：

A->B->**E**->D->**C**->F



自己实现LinkedList

虽然在实际开发中很少有这个需求，但在学习阶段，自己尝试着实现一下JDK中的LinkedList集合的主要功能是很有帮助的。

示例MyLinkedList.java展示了一种典型的实现方式.....

实现自己的LinkedList

```
public class MyLinkedList<E> implements List<E> {  
  
    //链表中的节点  
    private class Node {  
        Node next;  
        E data;  
    }  
  
    //初始时, new一个头节点  
    private Node head = new Node();  
}
```

LinkedList是一个链表，因此，定义了一个内部类Node，代表链表节点。

当初始化MyLinkedList<E>实例时，内置一个头节点，作为后继添加节点的“附着点”，这是一种比较简单的实现方式。

按位置提取集中的特定元素

```
//按位置提取
@Override
public E get(int index) {
    Node n = head;
    int i = index;
    while (i-- >= 0) {
        n = n.next;
    }
    return n.data;
}
```

按位置定位链表元素，只能通过“从头开始数数”的方式，很笨拙，成本颇高~

节点的追加

//在尾部追加

@Override

```
public boolean add(E e) {  
    Node newNode = new Node();  
    newNode.data = e;  
    //查找最后一个节点  
    Node last = head;  
    while (last.next != null) {  
        last = last.next;  
    }  
    last.next = newNode;  
    return true;  
}
```

//在中间追加

@Override

```
public void add(int index, E element) {  
    Node newNode = new Node();  
    newNode.data = element;  
    Node before = head;  
    int i = index - 1;  
    while (i-- >= 0) {  
        before = before.next;  
    }  
    Node after = before.next;  
    before.next = newNode;  
    newNode.next = after;  
}
```



如果给类新增一个引用链表尾的字段，在尾部追加元素就会快很多，这个任务留作练习。

修改或删除特定位置的元素

```
@Override
public E set(int index, E element) {
    Node n = head;
    int i = index;
    while (i-- >= 0) {
        n = n.next;
    }
    E old = n.data;
    n.data = element;
    return old;
}
```

```
@Override
public E remove(int index) {
    Node before = head;
    int i = index - 1;
    while (i-- >= 0) {
        before = before.next;
    }
    Node thisOne = before.next;
    Node after = thisOne.next;
    before.next = after;
    return thisOne.data;
}
```



时间主要花在定位上。

动手动脑



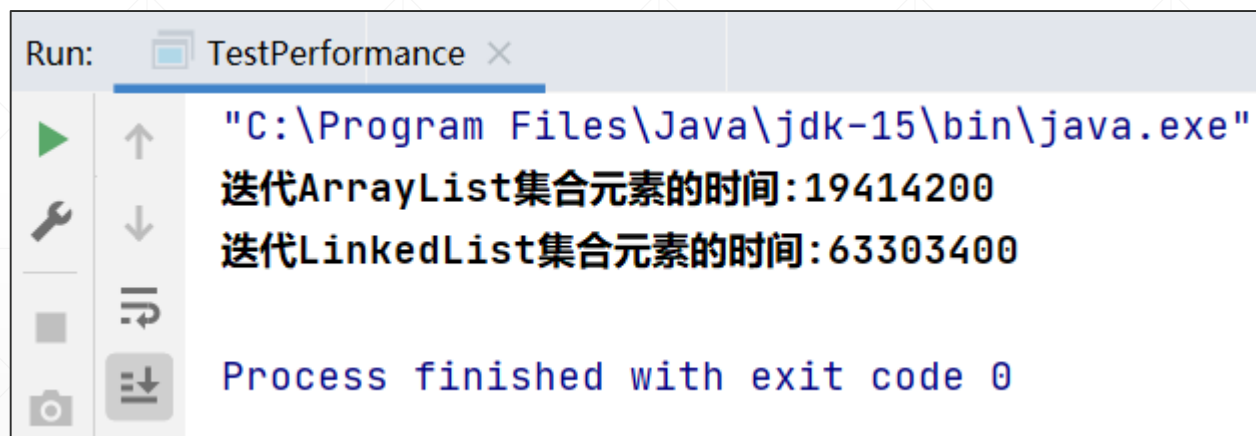
与MySimpleArrayList实例一样，MyLinkedList同样有大量的由List<E>所定义的方法需要实现。



JDK中提供了一个AbstractSequentialList抽象类，可用于快速定义和实现一个LinkedList，请你仔细阅读其文档和源码，然后使用它来定义一个LinkedList。

性能的比较

示例TestPerformance.java展示了迭代访问包容有900000个元素ArrayList与LinkedList所耗费的时间：



```
Run: TestPerformance x
"C:\Program Files\Java\jdk-15\bin\java.exe"
迭代ArrayList集合元素的时间:19414200
迭代LinkedList集合元素的时间:63303400
Process finished with exit code 0
```



可以看到，ArrayList的迭代遍历，速度比链表快得多，原因很简单，ArrayList内部是一个数组，数组的访问总是快的。

给LinkedList添加大量元素，不同方法性能差异极大！

示例ListInsertComparisons.java展示出对于LinkedList，使用LinkedList.add方法添加与使用ListIterator添加，虽然功能一样，但性能有着巨大的差异！

```
ListInsertComparisons x
"C:\Program Files\Java\jdk-15\bin\java.exe" --module-path "C:\Pr
Time to insert 250K in middle of linked list    41.623465 secs
Time to insert 250K using ListIterator         0.073993 secs

Process finished with exit code 0
```



插入大量的数据，使用迭代器，性能提升明显！



使用迭代器删除大量数据，是不是也有类似的特性？请同学们编写测试代码，给出一个结论。

小结



关于使用List集合有如下建议:

1

如果需要随机访问List集合元素, 应该使用ArrayList集合, LinkedList在这块表现不佳;
如果需要频繁地增删元素, 使用LinkedList, 它的速度很快。

2

如果经常要向已经包含大量数据的List集合执行插入操作, 强烈推荐
使用LinkedList集合。在这种场景下使用ArrayList, 性能受损严重,
因为需要时不时地重新创建一个新的数组, 并伴随有大量的元素复制。
另外, 实验表明: 使用迭代器完成LinkedList的插入操作, 非常快。